AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

SCHOOL OF COMPUTING

M M VASANTH

CH.SC.U4CSE24227

Week – 7

Design and Analysis of Algorithm(23CSE211)

1. RED BLACK TREE

Code:

```
#include <stdio.h>
#include <stdlib.h>

enum Color {RED, BLACK};

struct Node {
    int data;
    int color;
    struct Node *left, *right, *parent;
};

struct Node* root = NULL;

void leftRotate(struct Node *x) {
    struct Node *y = x->right;
    x->right = y->left;
    if (y->left != NULL) y->left->parent = x;
    y->parent = x->parent;
    if (x->parent == NULL) root = y;
    else if (x == x->parent->left) x->parent->left = y;
    else x->parent->right = y;
    y->left = x;
    x->parent = y;
}

void rightRotate(struct Node *x) {
    struct Node *y = x->left;
    x->left = y->right;
    if (y->right != NULL) y->right->parent = x;
    y->parent = x->parent;
```

```
    if (x->parent == NULL) root = y;
    else if (x == x->parent->right) x->parent->right = y;
    else x->parent->left = y;
    y->right = x;
    x->parent = y;
}

void fixViolation(struct Node *pt) {
    struct Node *parent_pt = NULL;
    struct Node *grand_parent_pt = NULL;

    while ((pt != root) && (pt->color != BLACK) && (pt->parent->color
== RED)) {
        parent_pt = pt->parent;
        grand_parent_pt = pt->parent->parent;

        if (parent_pt == grand_parent_pt->left) {
            struct Node *uncle_pt = grand_parent_pt->right;
            if (uncle_pt != NULL && uncle_pt->color == RED) {
                grand_parent_pt->color = RED;
                parent_pt->color = BLACK;
                uncle_pt->color = BLACK;
                pt = grand_parent_pt;
            } else {
                if (pt == parent_pt->right) {
                    leftRotate(parent_pt);
                    pt = parent_pt;
                    parent_pt = pt->parent;
                }
                rightRotate(grand_parent_pt);
                int t = parent_pt->color;
                parent_pt->color = grand_parent_pt->color;
```

```
grand_parent_pt->color = t;
                pt = parent_pt;
            }
        } else {
            struct Node *uncle_pt = grand_parent_pt->left;
            if ((uncle_pt != NULL) && (uncle_pt->color == RED)) {
                grand_parent_pt->color = RED;
                parent_pt->color = BLACK;
                uncle_pt->color = BLACK;
                pt = grand_parent_pt;
            } else {
                if (pt == parent_pt->left) {
                    rightRotate(parent_pt);
                    pt = parent_pt;
                    parent_pt = pt->parent;
                }
                leftRotate(grand_parent_pt);
                int t = parent_pt->color;
                parent_pt->color = grand_parent_pt->color;
                grand_parent_pt->color = t;
                pt = parent_pt;
            }
        }
    }
    root->color = BLACK;
}

void insert(const int data) {
    struct Node *pt = (struct Node*)malloc(sizeof(struct Node));
    pt->data = data;
```

```
pt->color = RED;
    pt->left = pt->right = pt->parent = NULL;




    if (root == NULL) {
        pt->color = BLACK;
        root = pt;
        return;
    }

    struct Node *curr = root, *parent = NULL;
    while (curr != NULL) {
        parent = curr;
        if (pt->data < curr->data) curr = curr->left;
        else curr = curr->right;
    }
    pt->parent = parent;
    if (pt->data < parent->data) parent->left = pt;
    else parent->right = pt;

    fixViolation(pt);
}

void inorder(struct Node *n) {
    if (n == NULL) return;
    inorder(n->left);
    printf("%d(%s) ", n->data, n->color == RED ? "R" : "B");
    inorder(n->right);
}
```

```
int main() {
    int n, val;
    printf("Name: M M VASANTH\n");
    printf("Roll No: CH.SC.U4CSE24227\n\n");
    printf("Enter number of elements to insert in RB Tree: ");
    scanf("%d", &n);
    printf("Enter elements:\n");



for(int i = 0; i < n; i++) {
        scanf("%d", &val);
        insert(val);
    }
    printf("Inorder Traversal (Sorted with Colors): \n");
    inorder(root);
    printf("\n");
    return 0;
}
```

Output:

```
vasanth@VasanthMM:/mnt/c/Users/vasan$ nano rbt.c
vasanth@VasanthMM:/mnt/c/Users/vasan$ gcc rbt.c -o rbt
vasanth@VasanthMM:/mnt/c/Users/vasan$ ./rbt
Name: M M VASANTH
Roll No: CH.SC.U4CSE24227

Enter number of elements to insert in RB Tree: 4
Enter elements:
13
56
64
62
Inorder Traversal (Sorted with Colors):
13(B) 56(B) 62(R) 64(B)
vasanth@VasanthMM:/mnt/c/Users/vasan$
```

⍰ Time Complexity: O(log N)

Because the tree ensures no path is more than twice as long as any other, the height is strictly bounded by O(log N). Insertion, deletion, and search all take O(log N) time in both average and worst cases.

⍰ Space Complexity: O(N)

Memory is required for the N nodes.

2. QUICK SORT WITH THREE DIFFERENT PIVOTS

Code:

```
#include <stdio.h>


void swap(int* a, int* b) {
    int t = *a; *a = *b; *b = t;
}


// Standard partition using the LAST element as pivot
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}


// 1. Pivot = Last Element
void quickSortLast(int arr[], int low, int high) {
    if (low < high) {
```

```
        int pi = partition(arr, low, high);

        quickSortLast(arr, low, pi - 1);

        quickSortLast(arr, pi + 1, high);

    }

}


// 2. Pivot = First Element

void quickSortFirst(int arr[], int low, int high) {

    if (low < high) {

        swap(&arr[low], &arr[high]); // Move first to last, then
partition

        int pi = partition(arr, low, high);

        quickSortFirst(arr, low, pi - 1);

        quickSortFirst(arr, pi + 1, high);

    }

}


// 3. Pivot = Middle Element

void quickSortMid(int arr[], int low, int high) {

    if (low < high) {

        int mid = low + (high - low) / 2;

        swap(&arr[mid], &arr[high]); // Move mid to last, then
partition

        int pi = partition(arr, low, high);

        quickSortMid(arr, low, pi - 1);

        quickSortMid(arr, pi + 1, high);
```

```
 }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr1[] = {10, 7, 8, 9, 1, 5};
    int arr2[] = {10, 7, 8, 9, 1, 5};
    int arr3[] = {10, 7, 8, 9, 1, 5};
    int n = 6;

    printf("Name: M M VASANTH\n");
    printf("Roll No: CH.SC.U4CSE24227\n\n");

    quickSortLast(arr1, 0, n - 1);
    printf("Sorted using LAST element as pivot: ");
    printArray(arr1, n);

    quickSortFirst(arr2, 0, n - 1);
    printf("Sorted using FIRST element as pivot: ");
    printArray(arr2, n);
```

```
quickSortMid(arr3, 0, n - 1);


printf("Sorted using MIDDLE element as pivot: ");
    printArray(arr3, n);


    return 0;
}
```

Output:

```
vasanth@VasanthMM:/mnt/c/Users/vasan$ nano qp.c
vasanth@VasanthMM:/mnt/c/Users/vasan$ gcc qp.c -o qp
vasanth@VasanthMM:/mnt/c/Users/vasan$ ./qp
Name: M M VASANTH
Roll No: CH.SC.U4CSE24227

Sorted using LAST element as pivot: 1 5 7 8 9 10
Sorted using FIRST element as pivot: 1 5 7 8 9 10
Sorted using MIDDLE element as pivot: 1 5 7 8 9 10
vasanth@VasanthMM:/mnt/c/Users/vasan$
```

⏱ Time Complexity: O(N * N)

Choosing the middle or a random pivot greatly reduces the chance of hitting the O(N^2) worst-case time complexity, which occurs when the array is already sorted and the first/last element is consistently picked as the pivot.

🧠 Space Complexity: O(log N)

In-place sorting, but recursive stack space takes logarithmic depth.

3. QUICK SELECT

Code:

```
#include <stdio.h>

void swap(int* a, int* b) {
    int t = *a; *a = *b; *b = t;
}

int partition(int arr[], int l, int r) {
    int x = arr[r], i = l;
    for (int j = l; j <= r - 1; j++) {
        if (arr[j] <= x) {
            swap(&arr[i], &arr[j]);
            i++;
        }
    }
    swap(&arr[i], &arr[r]);
    return i;
}

int quickSelect(int arr[], int l, int r, int k) {
    if (k > 0 && k <= r - l + 1) {
        int index = partition(arr, l, r);

        if (index - l == k - 1)
            return arr[index];
        if (index - l > k - 1)
            return quickSelect(arr, l, index - 1, k);

        return quickSelect(arr, index + 1, r, k - index + l - 1);
    }
    return -1;
}

int main() {
    int arr[] = {10, 4, 5, 8, 6, 11, 26};
    int n = sizeof(arr) / sizeof(arr[0]);
    int k = 3;

    printf("Name: M M VASANTH\n");
    printf("Roll No: CH.SC.U4CSE24227\n\n");
    printf("Array: 10 4 5 8 6 11 26\n");
    printf("The %d-th smallest element is %d\n", k, quickSelect(arr,
```

```
0, n - 1, k));


    return 0;
}
```

Output:

```
vasanth@VasanthMM:/mnt/c/Users/vasan$ nano qs.c
vasanth@VasanthMM:/mnt/c/Users/vasan$ gcc qs.c -o qs
vasanth@VasanthMM:/mnt/c/Users/vasan$ ./qs
Name: M M VASANTH
Roll No: CH.SC.U4CSE24227

Array: 10 4 5 8 6 11 26
The 3-th smallest element is 6
vasanth@VasanthMM:/mnt/c/Users/vasan$
```

Time Complexity: O(N * N)

Since we only recurse on one half of the array, the time reduces drastically compared to sorting the whole array. The worst-case is O(N^2) if bad pivots are chosen consistently.

Space Complexity: O(log N)

Recursive stack depth.